Accounting Integrity in Metered LLM-Serving Infrastructure: Verified Conditions Against Client-Side Under-Payment

Hardik[a,b,*], Natasha Kaila[b]

[a]Department of Computer Science and Data Analytics, Indian Institute of Technology Patna, India
[b]Vivekananda Institute of Professional Studies – Technical Campus, New Delhi, India

Abstract

Large language models are increasingly delivered as metered, network-accessible services in which value is streamed to the client before the charge is settled, usage is produced at runtime, and settlement is often asynchronous. We study the accounting integrity of such serving infrastructure against an authenticated client that seeks to pay less than it owes. Modeling a metered request as a lifecycle with a single integrity property, $V(r) \leq N(r)$, we localize failures to three points: state synchronization (B0), commitment timing (M1), and usage authority (M2). We verify the lifecycle exhaustively in TLA+/TLC across ten configurations and four invariants, and validate it on a testbed spanning three execution topologies, three accounting families including an asynchronous event pipeline, and a third-party serving stack. Three results follow. Reservation and abort-safe finalization are orthogonal: each secures only one of solvency and accounting integrity, and only their conjunction secures both. Computing usage correctly is not billing from it: an architecture that recounts accurately but charges the client-declared value leaks 58.3% of delivered value in a controlled setting and 69.0% against a real serving stack, while server-authoritative billing eliminates the leak. Asynchronous settlement reproduces over-serving under strictly sequential arrivals, so the synchronization condition concerns not concurrency but whether authorization is atomically coupled to economic commitment on the authorizing path; this persists under a network-separated two-machine deployment, while a settlement-confirmed serialization control shows none. We state model-level conditions separately from implementation patterns, report the cost of the strongest M2 defense, under 0.1% of end-to-end time with real inference, and delimit the formal model, which we extend with a second, causal-ordering specification of the asynchronous mechanism.

Keywords: LLM serving infrastructure, usage metering, distributed accounting, asynchronous settlement, formal verification, TLA+, cloud service billing integrity

1. Introduction

Metered computational services sell work whose quantity is not known when the work is authorized. A client asks for computation, the service performs and delivers it, and only afterwards does the system learn what to charge. Between those two moments the service holds economic state that is authoritative for billing but not yet durable, and that state is read, written, and reconciled by several components: an authorization path, an execution path that delivers value incrementally, and a settlement path that may run asynchronously behind both. Whether the service can be relied on to charge for what it delivered is therefore a question about distributed state management, not about accounting policy.

LLM inference serving makes the question concrete and current. The billed unit, the token, is produced at runtime by an opaque, streamed, long-running computation delivered to a remote client over a network; the quantity is a multi-category vector; and the usage record that determines the charge is itself produced by the serving system while the response is still in flight – so the authorization, execution, and settlement paths above now carry security-critical state. Mistakes in how that state is partitioned across components can produce integrity failures even when every individual component computes usage correctly, which is why this paper studies the accounting lifecycle itself: its states, its transitions, the conditions under which it is safe, and what happens empirically when those conditions are broken. Existing security work on this surface runs in three directions, all of which assume the client is honest, impersonated, or irrelevant. Provider-side auditing asks whether a dishonest provider can inflate hidden token counts [1, 2, 3]. Economic-abuse work asks whether a third party can drain a victim's budget [4, 5, 6, 7]. Attestation work asks whether an intermediary can misroute requests or forge provenance [8, 9]. The remaining quadrant is a legitimate, authenticated client that under-pays, and that is what we study (Table 1).

The individual mechanisms have precedents in isolation, surveyed in Section 3 and, for five independently developed metering gateways, collected as public reports in Section 3.5; none of those systems was tested by us, and none is claimed here to be exploitable. What has not been available is a single model that places them on one accounting lifecycle, checks it exhaustively, and states precisely when each defense is sufficient; that unification, verification, and empirical validation is the contribution

*Corresponding author.
Email addresses: hardik_24a12res263@iitp.ac.in (Hardik), 09617711924_aids@vipstc.edu.in (Natasha Kaila)

of this paper. Accordingly, B0 is presented as a known baseline and M1 and M2 as systematization and measurement contributions.

These failures are usually discussed as unrelated bugs in unrelated systems; placed on one request lifecycle, they are the same integrity property violated at three points, which yields three things no bug report does: defenses separate into properties that do not substitute for one another, the exploitable attack set becomes a function of the pricing basis rather than attacker ingenuity, and the conditions under which each defense suffices can be stated and checked.

The setting throughout is a network-accessible LLM serving system reached over a metered API, where a security-relevant decision, what to charge, is made from state produced while a response is still streaming. The question we ask is which conditions on the authorization path, the settlement path, and the billing input guarantee that delivered value never exceeds net committed debit, and how those conditions behave when settlement is distributed and asynchronous; Section 5 details why LLM serving exhibits every relevant ingredient at once. The conditions we arrive at are stated over the accounting lifecycle, not the model.

Contributions.

1. An accounting-state model of a metered service request (Section 4) connecting execution, value delivery, authorization, reservation, debit, reconciliation, and refund, with a single integrity property, $V(r) \leq N(r)$, that admits legitimate refunds. The model is stated over the request lifecycle of a metered service and is not specific to language-model inference.
2. A taxonomy of three experimentally distinguishable failure dimensions (Section 5) – synchronization, commitment timing, and usage authority – each localized to a distinct lifecycle element (a shared-state transition, a terminal path, a data-authority boundary), with known, systematized, and defensive boundaries stated explicitly.
3. Machine-checked verification of the lifecycle (Section 7) in TLA+, checked exhaustively with TLC over ten configurations and four invariants, producing counterexamples that match the measured failures and an exhaustive orthogonality result for the M1 defenses.
4. A distributed-settlement result (Section 10.4): deferring settlement reproduces the synchronization failure with strictly sequential arrivals and no concurrency at all, which forces a strictly more general condition than the one we started with, namely that authorization be atomically coupled to economic commitment on the authorizing path; the condition persists under a network-separated two-machine deployment, while a settlement-confirmed serialization control shows none (Section 10.5).
5. Empirical validation (Sections 8 and 10) across three execution topologies, three accounting families including an asynchronous event pipeline, two independent inference data planes, and a network-separated deployment, with fail-closed integrity gates and independent recomputation.
6. Sufficiency conditions and the implementation patterns that satisfy them, with their integrity and performance cost (Sections 9 and 11), kept explicitly separate, together with public evidence (Section 3.5) that the same architectural ingredient appears in independently developed, deployed metering gateways.

2. Threat Model and Security Property

The provider is honest. The adversary is a client with valid credentials whose goal is to receive metered inference and pay less than it owes. It controls six things, each exercised by our harness: request payloads, including any client-declared usage fields; permitted headers; request concurrency; the connection lifecycle, so it may abort at any moment; retries; and observation of its own responses and balance. It cannot compromise TLS, reach the database or Redis, read the gateway filesystem, compromise the inference backend, forge authenticated upstream metadata, or modify server code. This is an ordinary application-layer adversary, not a symbolic cryptographic one – every leak we report is attributable to the provider's own accounting design, not a compromise.

2.1. Attacker knowledge

Our M2 harness builds each declared-usage manipulation by transforming the true server-side usage vector, which hands the attacker more information than a real client has. Rather than leave that convenience implicit, we grade every manipulation by the knowledge it actually needs. K0 is client-observable only: the prompt sent, the response received, and the client's own balance. K1 additionally requires provider-reported usage, whatever the architecture returns. K2 requires oracle access to the exact server-side usage vector.

Under-reporting output or input, shaving a small constant, and declaring an inconsistent total are all K0, since a client can count its own prompt and the tokens it received. Inflating the cached-input category is K1, because the cached/uncached split is not visible in the response. Dropping hidden reasoning tokens is K2, because that count is never exposed. The headline usage-authority result rests on a K0 manipulation and does not depend on oracle knowledge; the two non-K0 manipulations are labeled and kept out of the main claim.

2.2. Integrity property

For a request r, let $V(r)$ be the value actually delivered, $D(r)$ the committed debits, $F(r)$ the refunds, and $N(r) = D(r) - F(r)$ the net committed debit. Under-payment and the integrity property are

$$\text{Leak}(r) = V(r) - N(r), \qquad \text{Integrity}(r): \; V(r) \leq N(r). \quad (1)$$

The inequality is over the net debit so that reconciliation remains legitimate: a client that consumes less than its reservation must get the difference back. A refund is legitimate only when it releases the unconsumed portion,

$$F(r) \leq \text{Res}(r) - V(r), \quad (2)$$

Table 1: Adversary directions in LLM inference accounting. Prior work covers the first three rows; this paper studies the fourth.

Adversary	Goal	Representative work	Studied here
Provider	Over-charge an honest client	CoIn [1]; Invisible Tokens [2]; Token Inflation [3]	no
Third party	Inflate a victim's bill	Denial-of-Wallet [4, 5]; OWASP LLM10 [6]; LLMjacking [7]	no
Intermediary	Misroute or forge provenance	AEX [8]; gateway-path provenance [9]	no
Client	**Under-pay for received inference**	—	**yes**

and the vulnerable "refund the whole reservation on abort" architecture violates exactly this side condition.

2.3. Two properties, not one

Accounting integrity is not the only thing a metering system owes, and merging the two obscures the defense analysis. We separate them:

- *Accounting integrity*, $V(r) \leq N(r)$: the client is never net-charged less than the value it received.
- *Solvency*: a request is served only when the system holds enough committed or reserved capacity to cover it under its own policy.

These are independent. A system can bill exactly for what it delivered while serving clients who cannot pay, and it can hold funds correctly and then hand them back. In the M1 dimension, reservation addresses solvency and abort-safe finalization addresses accounting integrity. Section 8.2 shows measurement and Section 7 shows exhaustive checking that neither substitutes for the other.

3. Background and Related Work

LLM APIs meter multi-category token vectors, stream output incrementally, and settle charges after the fact, each with its own literature; the gap we occupy sits between them. The study also draws on the transaction and consistency literature that governs shared economic state, the escrow-style reservation pattern for high-contention counters, and the verification practice used to check stateful service designs.

3.1. Prior adversary framings

Pay-as-you-go pricing is a defining feature of cloud services [16], enforced by deciding whether to admit work, observing usage, and committing a charge – a placement of responsibilities that determines security [17]; the failures studied here follow from where that placement sits in the request path, not from a defect in any single component. Existing work covers the other three quadrants of Table 1. CoIn [1] shows reasoning tokens invisible to the client can be counted and audited only with effort; Sun et al. [2] argues opaque LLM services need audit mechanisms because the billed quantity is unverifiable to the payer; and Hoque et al. [3] measure how far a dishonest provider can overcharge before detection becomes likely – all three take the client's side against the provider, a direction we invert while keeping their observation that the billed quantity is runtime-produced and hard to verify. A second group inflates somebody else's bill: Kelly et al. [4] define forced financial exhaustion as a DoS-adjacent threat in serverless settings, consolidated by Dorsett et al. [5]; OWASP [6] ranks unbounded consumption among the top LLM-application risks; Sysdig Threat Research Team [7] document stolen credentials resold for inference at the victim's expense; and Shumailov et al. [18] show inference cost can be driven adversarially. Logic flaws in payment flows are related: cashier-as-a-service lets shoppers obtain goods without paying [14], and concurrency attacks on database-backed applications exploit weak isolation for financial effect [15], but our billed quantity is produced during the request rather than fixed at checkout; complete mediation and fail-safe defaults [19] apply directly, and the fail-closed recount path in Section 8 follows the second. Attestation and provenance work targets the gateway path, not the endpoint: Guan [8] provide multi-hop attestation for LLM APIs, and Wang and Tian [9] address forged routing claims with evidence-bound provenance, assuming a dishonest intermediary rather than endpoint.

3.2. Generic mechanisms

Relying on untrusted input in a security decision is CWE-807 [10]. Concurrent quota races are well-characterized, and Kettle [13] shows how narrow the exploitable window can be; the escrow transactional method [20] is the classical treatment of high-contention counter updates and ancestor of the reserve-and-reconcile pattern we evaluate. Streaming and client cancellation are specified behavior: server-sent events [21] define a channel the client may close at any time, and HTTP semantics [22] leave an aborted response's consequences to the application.

B0 is a lost update per critiques of the ANSI isolation levels [23], sharpened by implementation-independent isolation definitions [24]. Reserve-and-reconcile with a compensating refund resembles sagas [25], and our asynchronous family's idempotent, at-least-once handling follows standard messaging idempotence [26]. Bailis et al. [27] analyzes whether an invariant like a non-negative balance needs coordination to maintain; an authorization check against a bound does – the B0 condition – in the spirit of linearizability [28]. The delay between value delivery and durable debit in the asynchronous family instances the staleness eventually consistent stores expose [29], treated practically by Kleppmann [30].

3.3. Serving and tokenization

The billed quantity depends on a tokenizer: subword segmentation [31] and SentencePiece [32] underlie today's tok-

enizers, and engines legitimately disagree on the token count for identical text. Modern serving systems batch and cache aggressively: PagedAttention/vLLM [33] established the prefix-reuse memory management that makes the cached-token category observable and unstable in Section 10, and keep changing execution structure – chunked prefill [37], disaggregated prefill/decoding [38], structured-program prefix reuse [39], and speculative decoding [40] – so a metering subsystem cannot assume one fixed execution model. Token counts also depend on tokenizer and language [41]. Our real-stack validation uses llama.cpp [34] serving SmolLM2-135M-Instruct [35], with tiktoken [36] among the tokenizers benchmarked.

3.4. Verification and validation practice

We specify the lifecycle in TLA+ [42, 43] and check it with TLC [44], in the model-checking tradition [45, 46, 47]. Using model checking to find architectural defects that testing misses is established industrial practice [48], and our experience matches the reported pattern: the checker's value lay in refuting a specification we believed, not confirming one. Surveys of formal-methods adoption [49, 50] and Lamport's case for specifying before coding [51] motivate the practice; implementation-level verification has been achieved for an OS kernel [52], a compiler [53], and distributed systems [54] at substantial proof effort, while lightweight methods validate against executable reference models [55] – an empirical study found bugs concentrated where a verified specification meets unverified components [56], the gap our evidence labels keep explicit. Property-based testing [57] and metamorphic testing [58, 59] check properties and relations between outputs without an oracle, as our price-scaling invariance check does; the oracle problem [60] is why we re-derive every reported quantity independently rather than trust the system's own bookkeeping. Fault injection is established practice: chaos engineering perturbs running systems [61], lineage-driven injection guides fault selection [62], and Elle checks recorded histories for isolation anomalies [63]; our nine lifecycle failures are enumerated rather than guided, and real cloud-outage studies [64] motivate treating abnormal terminal paths as first-class test targets.

3.5. Evidence from deployed systems

The mechanisms studied here are not confined to our testbed: Table 2 lists public, independently filed reports from five separately developed metering gateways and proxies showing the same architectural ingredient, reviewed directly from the linked issue and pull-request pages, with each row stating what the report does and does not establish.

This evidence is used carefully: we claim no listed system is exploitable and attempted nothing against any of them – every measurement here comes from our own local testbed; presence of an architectural ingredient is not presence of a vulnerability, which is why the table separates the two columns; and two of the five reports are already fixed by their own maintainers, so they are historical evidence the failure mode occurs and is repairable, not a statement about those systems today. The table supports the weaker claim we need: these accounting-lifecycle gaps occur in independently written gateway code, not only in ours.

3.6. The gap

Within our targeted corpus we found no study evaluating client-side under-payment under an honest-provider threat model across representative metering architectures, with a common measurement and an ablation of the enforcement primitives. This is a bounded negative from a targeted search. It does not establish that no such work exists.

4. Accounting State Model

We model a metered inference request as a state machine (Figure 1) so that each mechanism can be localized to a specific transition, establishing that the taxonomy is architectural rather than a list of unrelated implementation bugs. Table 3 summarizes where each mechanism sits and what closes it.

4.1. Formal model

For a request r let $U_{\text{true}}(r)$ be the usage actually produced, $U_{\text{bill}}(r)$ the usage billed, and $P(\cdot)$ the exact-decimal pricing function, so that $C(r) = P(U_{\text{true}}(r))$ is the true cost. $V(r)$, $\text{Res}(r)$, $D(r)$, $F(r)$, and $N(r) = D(r) - F(r)$ are as defined in Section 2 (Equations 1–2). At trial level the harness independently verifies conservation, $\sum_{r \in T} N(r) = B_{\text{initial}} - B_{\text{final}}$, and fails the build on any divergence.

4.2. Mechanism localization

B0 (synchronization). The baseline implementation manifests as follows: if two concurrent requests read the same authorization state B and both commit a decrement derived from it, one decrement is lost, $\sum_i N(r_i) < \sum_i V(r_i)$. This particular manifestation requires two or more overlapping transactions, which is why the baseline leaks nothing at concurrency 1. The general failure condition is broader than this trigger, however: the defect occurs whenever authorization observes a state that does not yet reflect the economic commitments generated by other authorized requests. Concurrent overlap is one way to create that condition; Section 10.4 shows that deferred settlement creates the same condition under strictly sequential arrivals, with no overlap at all. We therefore distinguish, throughout this paper, the *baseline trigger* (concurrent overlap, in this implementation) from the *general security condition* (authorization not atomically coupled to economic commitment on the authorizing path).

M1 (commitment timing). If $V(r) > 0$ in STREAMING but the exit path taken (ABORTED) does not reach RECONCILED with $N(r) \geq V(r)$, then $\text{Leak}(r) = V(r) > 0$. The defect is in the transition graph, not in a shared resource: it is a per-request lifecycle property, and Section 8.2 confirms empirically that it does not depend on concurrency in the tested architecture.

Table 2: Public reports from independently developed metering gateways exhibiting the architectural ingredients modeled in this paper. No claim is made that any of these systems is exploitable; none was tested. Pages reviewed 23 September 2026.

System	Public report	Ingredient	What it establishes	What it does not establish
LiteLLM	Issue #14457, open [65]	Terminal path without a committed accounting event (M1)	That an independently developed gateway loses usage tracking when a client disconnects before the provider's final usage chunk	Exploitability, financial impact, or behaviour after the review date
OpenGateLLM	Issue #1148, fixed [66]	Terminal path without a committed debit (M1), with a measured instance	That $V(r) > N(r)$ was observed and quantified by the maintainers, who report 254,347 completion tokens delivered and never billed, and that it was repairable once identified	Representativeness of the instance, or that the system exhibits it now
CLIProxyAPI	Issue #2179, closed [67]	Usage record absent at the accounting transition despite delivered value	That usage-record loss also occurs on successful responses, not just aborts, broadening where the event can be missed	Root-cause parity with our taxonomy; the fix is not analysed here
new-api	Issue #5235 [11]	Refund of pre-consumed quota when the final usage record never arrives (M1)	That reserve-and-refund handling of disconnects is a real design point, not a constructed one	That the refund path is exploitable as reported
Aperture	Merged pull request #247 [12]	Guard against an encoding-induced silent zero-debit (M2 direction)	That server-side authority over billed quantity is treated as a defect worth a shipped mitigation	That the mitigation is complete or that the prior behaviour was exploited

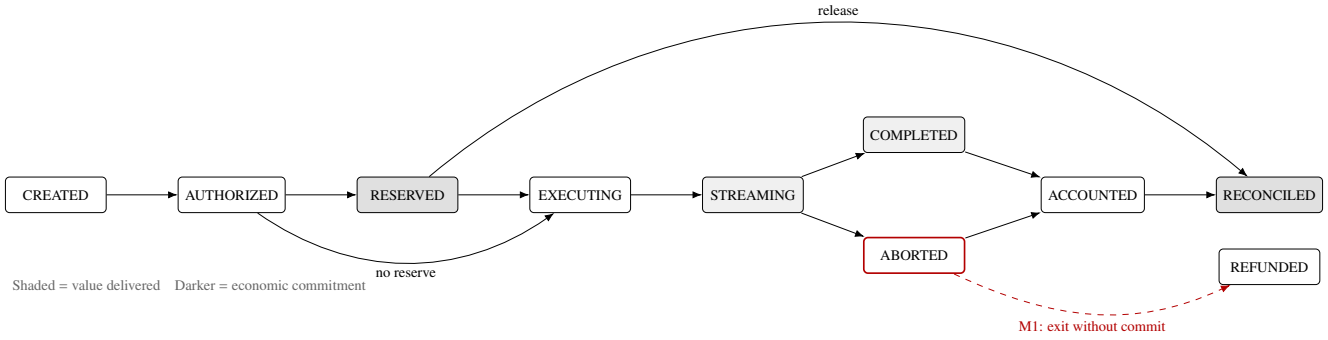


Figure 1: Request lifecycle. Value exposure begins at the first delivered token (STREAMING); economic commitment is final only at RECONCILED. The interval between them is the exposure window that M1 exploits. B0 corrupts the AUTHORIZED → RESERVED transition (non-atomic); M2 corrupts the data used at ACCOUNTED.

M2 (usage authority). If the billing basis is client-controlled, $U_{\text{bill}} = T(U_{\text{true}})$ for an attacker transformation T, then $\text{Leak}(r) = P(U_{\text{true}}) - P(T(U_{\text{true}})) > 0$ whenever $P \circ T < P$. Authority is a per-request data-provenance property; Section 8.3 shows this is analytically independent of concurrency because the billed amount never reads shared mutable state.

Read architecturally, the three mechanisms sit at three different kinds of boundary. B0 concerns state ownership on the authorization path: which component reads the balance, and whether that read and the economic commitment are one atomic step. M1 concerns the settlement path: whether every terminal path of the request lifecycle reaches a committed accounting event. M2 concerns data authority: which representation of usage the billing function reads. Solvency and accounting integrity are separated in M1 because different steps of the lifecycle own them, reservation before execution and finalization on every exit path.

Section 8 confirms this prediction empirically: the B0 manifestation we implemented requires concurrency; M1 and M2 per-request leakage does not.

5. Architectural Taxonomy

The three dimensions just distinguished architecturally are also experimentally distinguishable, not merely conceptual: B0 is a non-atomic authorize-then-decrement on shared credit state, a known baseline used to validate the measurement rig and supply a generic reference point, while M1 and M2 remain the lifecycle- and authority-sensitive dimensions of Section 4.

Table 3: Structural distinction between the three mechanisms: what is defective, what triggers the baseline manifestation we implemented, and what closes it. "Trigger" is implementation-specific; the general condition (final column of Table 6) is what actually matters and is not concurrency-specific for any of the three.

Mechanism	Defective element	Baseline trigger (this implementation)	Closed by
B0	transition atomicity	overlapping requests (concurrency ≥ 2); deferred settlement also suffices (Sec. 10.4)	authorization atomically coupled to commitment on the authorizing path
M1	transition graph (abort path)	any single abort, no concurrency needed	abort-inclusive finalization
M2	data authority at ACCOUNTED	any single request with a client-controlled billing basis, no concurrency needed	server-authoritative recount

5.1. On LLM specificity

We do not claim these mechanisms are possible only in LLM systems. Streaming responses, trusting client-supplied values, and racing a counter all occur elsewhere. What is specific to LLM inference is that one request combines, at a single metering boundary: progressive value delivery; usage that is generated at runtime and therefore unknown at authorization time; multi-category token vectors; tokenizer- and provider-dependent usage semantics; pricing functions defined over those categories; and asynchronous settlement. It is the combination we systematize, not the individual pieces.

The three dimensions generalize to different degrees, stated rather than blurred. B0 is fully generic: a shared-counter race that behaves the same in any metered service, and the revised condition of Section 10.4 – that authorization be atomically coupled to economic commitment on the authorizing path – is a statement about distributed accounting state with no reference to inference at all. M1 generalizes to any service that delivers value progressively over a long-running request, streamed media and batch jobs included, and is interesting exactly where a terminal path can bypass the accounting event. M2 is the narrowest: it needs a billed quantity that is runtime-produced, multi-category, and priced by a category-sensitive function, currently most visible in LLM APIs but not unique to them.

The boundary studied here is the usage-metering and accounting subsystem of a network-accessible serving system: an authorization path, a settlement path, and an accounting event lifecycle, with state ownership divided among the gateway, the accounting store, and, in the asynchronous family, an event pipeline, and a data-authority decision at ACCOUNTED about which representation of usage the billing function reads. We do not study the behavior, robustness, or output safety of the language model itself.

6. System and Experimental Methodology

The experiments validate the architectural behavior and engineering claims of the preceding sections – not a user study, industrial evaluation, or field deployment – and the study deliberately trades realism for control and precision [68]. The testbed is a FastAPI gateway over PostgreSQL 16 in READ COMMITTED and Redis 7, running under Docker Compose on one commodity host with 16 logical CPUs and 16 GB of RAM. Money is exact decimal throughout. Five M1 architectures and six M2 architectures sit behind common interfaces, so experiments compare semantics rather than unrelated code. The default generator is deterministic, which is what makes the accounting arithmetic exact; real tokenizers run on the recount path, and Section 10 replaces the generator entirely with third-party serving code.

Pricing is category-wise over three tiers. Leakage *efficiency*, the fraction of true value evaded, is the headline metric because it is invariant under uniform price scaling; absolute dollar figures are labeled as scenario values. The sweeps are: B0 concurrency $\{1, 2, 5, 10, 20, 30, 50\}$ with 30 repetitions; M1 across architecture, abort position, and concurrency $\{1, 5, 20, 50, 100\}$; M2 across architecture, manipulation, and concurrency. Request volume is held constant across concurrency levels, so that volume cannot masquerade as vulnerability.

6.1. Integrity gates

Every summarizer recomputes each record's leak from primitives, checks invariant-flag consistency, and verifies ledger conservation $\sum_r N(r) = B_{\text{init}} - B_{\text{final}}$ without trusting the application's own bookkeeping. A violation fails the build. The gates are validated adversarially, not assumed: corrupting a leak value, debit, served flag, balance, or invariant flag, and deleting a ledger row, must each be detected – and all twelve injections were. The metamorphic suite also caught one of our own unjustified assumptions. We had asserted that leakage efficiency is invariant across price tiers; it is, but only under uniform scaling, not under a structural change to the price ratios. We corrected the property rather than suppress the failure.

6.2. Independent verification

The strongest M2 defense is re-implemented in a module that shares no code path with the gateway, and the two agree on all cross-validated cells (42 M2, 16 M1, 2 B0). A separate audit re-derives every reported quantity from raw data using code that imports nothing from the project, and reports zero discrepancies across 4,800 M2 records, 2,400 M1 records, and 420 B0 trials. The independent checker initially reported three disagreements. They were definitional, not defects: the checker aggregated under-payment only, the gateway the signed billing delta, and under flat-total pricing an honest client is systematically overcharged, so the two aggregates legitimately differ. Both are now reported, and the security property uses under-payment.

6.3. Statistics

All statistics come from a standalone script using SciPy [69] and statsmodels [70], writing a single output file that the paper cites. Many conditions here are deterministic by construction and have exactly zero variance. For those we report the constant and the complete separation rather than a p-value, and reserve nonparametric tests and bootstrap intervals [71] for quantities that genuinely vary, in line with common guidance for the statistical analysis of randomized experiments in software engineering [72].

6.4. Reproducibility

The artifact is archived with a DOI, with raw data and reproduction scripts released alongside it. Repeatability of computer-systems research is known to be weak [73], and repeatability, reproducibility, and corroboration are distinct goals [74]; we address the first two by releasing code and raw data, not the third. The release is consistent in spirit with the FAIR principles for research data [75].

7. Formal Verification

Measurement shows a defect occurs on the schedules our harness produced; it cannot show a defended architecture is safe on schedules it did not produce. We therefore encode the lifecycle of Section 4 in TLA+ [42] and check it exhaustively with TLC 2.19 [44]. In software-verification terms, the specification is an executable architectural model against which each configuration's safety properties are checked; it verifies the model, not the implementation.

7.1. Model

One specification covers all three dimensions, with each dimension as a Boolean constant, so that the safe and unsafe architectures are the same state machine under different constants, exactly how the testbed implements them behind one interface. The constants are `AtomicDebit` for B0, `Reserves` and `AbortSafe` for M1, and `ServerAuthority` for M2. A fifth constant, `ClientMayAbort`, is a workload switch rather than an architecture property; disabling it in the B0 configurations keeps the synchronization counterexample free of commitment-timing noise, mirroring how the experiments hold the other dimensions at their safe setting.

Four invariants are checked, each the formal counterpart of a gate the summarizers already apply to every record: AccountingIntegrity ($V(r) \leq N(r)$ for terminal r), Solvency (balance ≥ 0), LedgerConservation (once all requests are terminal, $B_{\text{init}} - B_{\text{final}} = \sum_r N(r)$), and RefundBounded, which is Equation (2). Solvency is checked separately from accounting integrity; collapsing them would destroy the distinction the ablation exists to establish.

7.2. Method

Each invariant is checked in its own TLC run. TLC halts at the first violation it finds, so checking jointly would hide the fact that an M2 architecture breaks two of them. The result is a full configuration × invariant matrix (Table 4): ten configurations and four invariants give 40 independent runs over 27,526 distinct states. Every expected outcome is declared in the checking script before the run, and the script exits non-zero on any disagreement. All 40 matched, with no disagreements. Runs use a single worker deliberately: with parallel workers, a run that halts at a counterexample explores a nondeterministic number of states, and the totals drifted between runs.

Table 4: TLC model-checking results across ten architecture configurations and four invariants. All 40 outcomes matched the expectation declared before the run.

Configuration	Accounting integrity	Solvency	Ledger conservation	Refund bounded
b0 vulnerable	holds	holds	VIOLATED	holds
b0 hardened	holds	holds	holds	holds
m1 post completion	VIOLATED	holds	holds	holds
m1 reserve refund on abort	VIOLATED	holds	holds	VIOLATED
m1 no reserve settle	holds	VIOLATED	holds	holds
m1 reserve reconcile	holds	holds	holds	holds
m2 client	VIOLATED	holds	holds	VIOLATED
m2 server recount	holds	holds	holds	holds
all defenses	holds	holds	holds	holds
all defenses 3req	holds	holds	holds	holds

7.3. Counterexamples

TLC returns a shortest counterexample for each unsafe configuration, and each reproduces the mechanism we measured. For B0, the 21-step trace is the lost update exactly: both requests read the same balance, both write an absolute value derived from that stale read, and both then deliver and are charged. Which invariant fails is the interesting part. Accounting integrity and the refund bound both hold, because every individual record is internally consistent; what breaks is conservation of the shared balance. That is why a per-record audit cannot detect B0 while a conservation check can.

For M1, the model checks all four combinations of reservation and abort-safe finalization, not just the two the implementation ships. Reservation alone yields solvency and violates integrity. Abort-safe finalization alone yields integrity and drives the balance to -1. Only the conjunction satisfies both. The orthogonality claim is therefore established by exhaustive exploration rather than by the two architectures we happened to build.

For M2, the client-authoritative architecture violates accounting integrity and also the refund bound: with a reservation in place, under-billing surfaces as an over-refund. That the same defect appears in two independent invariants is a consequence of the model rather than something we set out to show, and it became visible only because the invariants are checked separately. Enabling server authority restores all four invariants against the same lying client; the declared value simply stops being read.

The three defenses also compose. With contention, aborts, and a lying client all enabled, no invariant is violated on any reachable state, at two and at three concurrent requests.

7.4. Evidence labels and their limits

We use three labels and do not interchange them. *Model-checked* means TLC explored the reachable state space exhaustively under a stated configuration. *Formally argued* means stated over the accounting model and reasoned about by hand. *Empirically verified* means measured and independently recomputed. Nothing weaker than the first is ever described as proved.

Nothing here shows that a production implementation is safe. The model abstracts away the database, the network, partial failure, asynchronous reconciliation, multi-category pricing, and tokenizer disagreement. It is finite, at two to three requests with unit pricing. Only safety properties are checked;

no liveness property is verified. **We do not provide a mechanized refinement proof from the abstract specification to the implementation.** The correspondence between the two is an argument supported by case-by-case agreement between formal and empirical outcomes, and we do not claim more.

7.5. Validating the specification

Declaring expectations before running earned its keep immediately. Our first specification treated a reservation as a hold and set the committed debit equal to the charge at settlement, so $N = D - F$ fell below the delivered value and TLC rejected the safe configurations. The model was wrong, not the paper: in a reserve-and-reconcile architecture the reservation *is* the committed debit, and the refund returns its unused part. Correcting the settlement semantics made the safe configurations verify. We record the failure because a formal model that is edited until it agrees with the paper is worth nothing.

7.6. A second model: the asynchronous-settlement race

The specification above assumes strongly consistent accounting state and is silent on the mechanism behind Section 10.4: deferred settlement, not concurrency, separating authorization from commitment. We built a second, separate TLA+ specification, `AsyncSettlementTOCTOU.tla`, rather than extending the first, because the two mechanisms are different in kind. It is a causal-ordering model, not a timed one: a request moves through CREATED → ADMITTED/REJECTED → SETTLEMENT_PENDING → SETTLED (or RELEASED), and interleavings are ordered by happens-before, not by milliseconds, matching the T1–T7 instrumentation's own finding that the authorization step is a plain read and only settlement ever writes the balance.

We checked it with TLC at two (N, k) points, $(6, 2)$, matching the primary B0-async workload exactly, and $(4, 2)$, matching a second tested empirical configuration. In the vulnerable configuration, SOLVENCY is violated at both points; TLC's shortest counterexample at $(6, 2)$ needs only three staggered, non-simultaneous admissions against a two-unit budget. ACCOUNTINGINTEGRITY and LEDGERCONSERVATION hold exhaustively at both points (re-checked in isolation, 91,921 states generated, 22,521 distinct, no violation). In the atomic configuration all three invariants hold at both points.

This is a useful contrast with Table 4, not a repetition of it: the synchronous model's vulnerable B0 configuration violates LEDGERCONSERVATION and holds SOLVENCY, the signature of a lost update; this model's vulnerable configuration violates SOLVENCY and holds LEDGERCONSERVATION, the signature of an admission race with no lost write at all. Two mechanistically distinct races leave different invariant fingerprints, which is itself a form of cross-check between the two specifications rather than noise to explain away.

The same evidence labels and limits apply as in Section 7's own account of itself: this is *model-checked*, exhaustively, at two finite (N, k) points, over an abstract, untimed model of the mechanism. It does not verify the implementation, a timed or networked behavior, or anything beyond the two configurations checked, and it is the formal counterpart of the empirical result in Section 10.5, not a substitute for it.

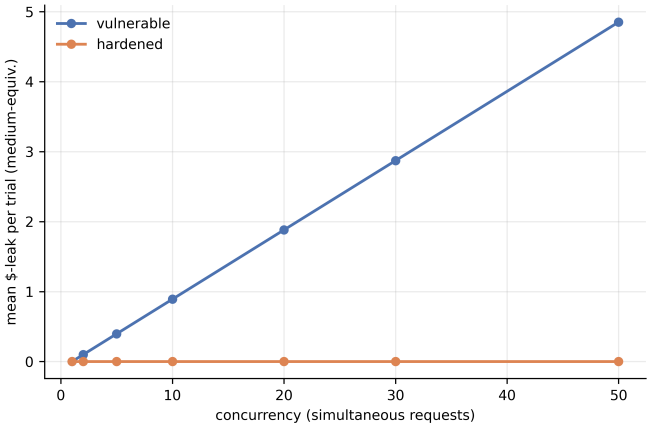


Figure 2: B0 baseline: mean leakage per trial against concurrency, 30 repetitions per cell, deterministic generator, growing linearly at 0.099 per additional concurrent request ($R^2 = 1.0000$). Section 10.4 shows concurrency is not the essential ingredient: delayed settlement reproduces over-serving with strictly sequential arrivals.

8. Empirical Results

8.1. B0: synchronization

The non-atomic check-then-decrement baseline leaks only when at least two requests overlap; this is a property of the implemented trigger, and Section 10.4 shows a different trigger for the same underlying condition. Mean leakage is \$0.000 at concurrency 1, then \$0.099, \$2.871, and \$4.851 at concurrency 2, 30, and 50, growing linearly at 0.099 per additional concurrent request with $R^2 = 1.0000$ and $p = 1.9 \times 10^{-50}$ (Figure 2). The hardened atomic compare-and-decrement leaks nothing at every level. B0 is not a contribution; it validates the rig and supplies the contrast used below. Section 10 revises the conclusion drawn from it.

8.2. M1: commitment timing

Leakage tracks the value delivered before the missing commit. It rises monotonically with abort position and collapses to zero at completion, when the debit finally fires: per request, \$0.0045 at roughly one token delivered, \$0.0285 at 25%, \$0.0540 at 50%, \$0.0960 at 90%, and \$0.0000 at completion (Figure 3). Request-level attack success is 1.00 at every abort position before completion. The two vulnerable architectures, debit only after completion and refund the whole reservation on abort, are economically identical.

8.2.1. Two orthogonal properties

The ablation separates solvency from accounting finality. Given a budget for two requests and six issued, the no-reserve-settle architecture preserves accounting integrity (leakage is \$0.0000 at every abort position) yet serves all six and drives the balance to −\$0.32. Conversely, the reserve-refund-on-abort architecture holds a reservation and then refunds it away, failing both. Table 5 states the resulting design rule; Section 7 establishes it exhaustively rather than by these two examples.

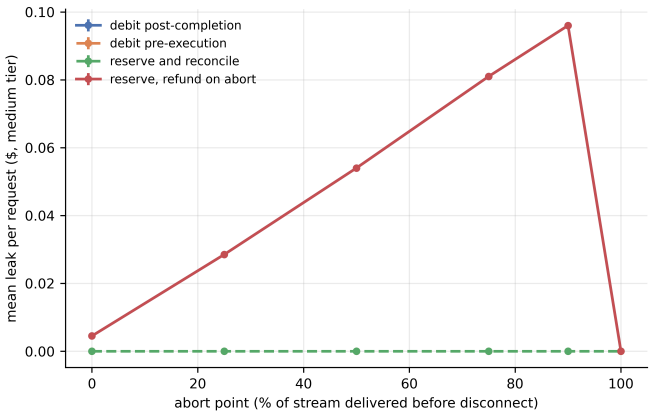


Figure 3: M1 leakage (delivered value minus net committed debit) against abort position, deterministic generator, concurrency 1, medium price tier, 20 requests per cell. Vulnerable architectures peak just before the commit would have fired and collapse to $0 at completion; safe architectures stay at $0 throughout. Error bars are within-cell standard deviations, zero at this concurrency.

Table 5: M1 orthogonality. Each control secures one property; only the conjunction secures both. All four cells are model-checked (Table 4); the diagonal cells are also measured.

	Reservation	
Abort-safe finalization	No	Yes
No	neither property	solvency only
Yes	integrity only	both

8.2.2. *Concurrency is not the mechanism*

With request volume held constant, per-request leakage was unchanged across concurrency levels 1 to 100: $0.0528 per request at every level, identical to four decimal places, with request-level attack success exactly 1.000 for vulnerable and 0.000 for safe architectures (Figure 4). The honest-client control leaked nothing anywhere. We do not claim invariance as a proven property of all systems under all loads; this is the measured behavior of the tested architecture, and Section 12 gives the structural reason the model predicts it.

8.3. *M2: usage authority*

The M2 result is best read as a chain. True usage is produced by the model; usage computation turns it into a vector; that vector is stored; a billing authority decides which representation to price; and a debit follows. A system can compute and store the correct usage while still billing from a different, attacker-influenced representation, and nothing in the chain flags the discrepancy.

That is not hypothetical. The client-logged architecture performs an accurate server-side recount, records it alongside the declared vector, and bills the declared number. Against 90% output under-reporting it leaks 58.3% of delivered value in the controlled setting, and 69.0% against the real serving stack of Section 10. The manipulation is K0: a client can count the tokens it received, so no privileged knowledge is required. Every server-authoritative architecture (server-recount, hybrid-reconcile, and upstream) drove leakage efficiency to 0.000 across all eight manipulations in the tested catalog (Figure 5).

Writing U for the true usage vector, P for the pricing function, and T for the attacker's transformation of declared usage, a manipulation pays off exactly when

$$P(T(U)) < P(U), \tag{3}$$

that is, when the pricing basis reads the field T changes. Section 11 develops the consequences. The exploitable set is a property of the billing function, not of attacker ingenuity.

M2's independence from concurrency is analytic rather than discovered. The billed amount is a pure function of one request's declared usage, computed before any balance is read, so no shared mutable state enters the billing decision and the leak of one request cannot depend on what else is in flight. The concurrency sweep confirms the derivation and serves as a check that the implementation contains no unintended shared-state dependency. It passes, and we present it as a much weaker claim than B0's measured slope.

8.4. *Lifecycle failure modes*

Beyond the scripted attacks we inject nine lifecycle failures: disconnect at $t = 0$ and mid-stream, truncated stream, settle-window race, missing, delayed, and duplicated usage metadata, and a recount-engine failure. Every invariant violation falls on a vulnerable architecture, the hardened architectures survive all of them, and a recount-engine failure is fail-closed: the request is rejected and nothing is served or billed.

8.5. *Limits of detectability measurement*

A natural extension of this measurement is whether a leak is detectable after the fact from retained evidence, distinguishing an invisible leak from one that is merely unreconciled. We tested this directly and found that our instrumentation cannot support the distinction.

Two defects were involved. The implementation assigned the detectability level from static architecture metadata rather than measuring it, so the experiment read back a label we had set. When we replaced that with a classifier reading only retained ledger evidence and blind to architecture labels, the distinction disappeared: the two architectures in question retain identical evidence, so both are equally reconcilable. Across the full dataset, 2,200 records carrying the stored "invisible" label are reconcilable or prevented on the evidence, and no record in the study qualifies as invisible at all.

The root cause is a property of the testbed rather than of the architectures: our harness records ground-truth usage for every architecture because it needs that value to compute leakage, whereas a production deployment need not retain it. Our instrumentation therefore cannot distinguish detectability classes, so we make no detectability claim. We report this as a limitation rather than build additional instrumentation to rescue the metric.

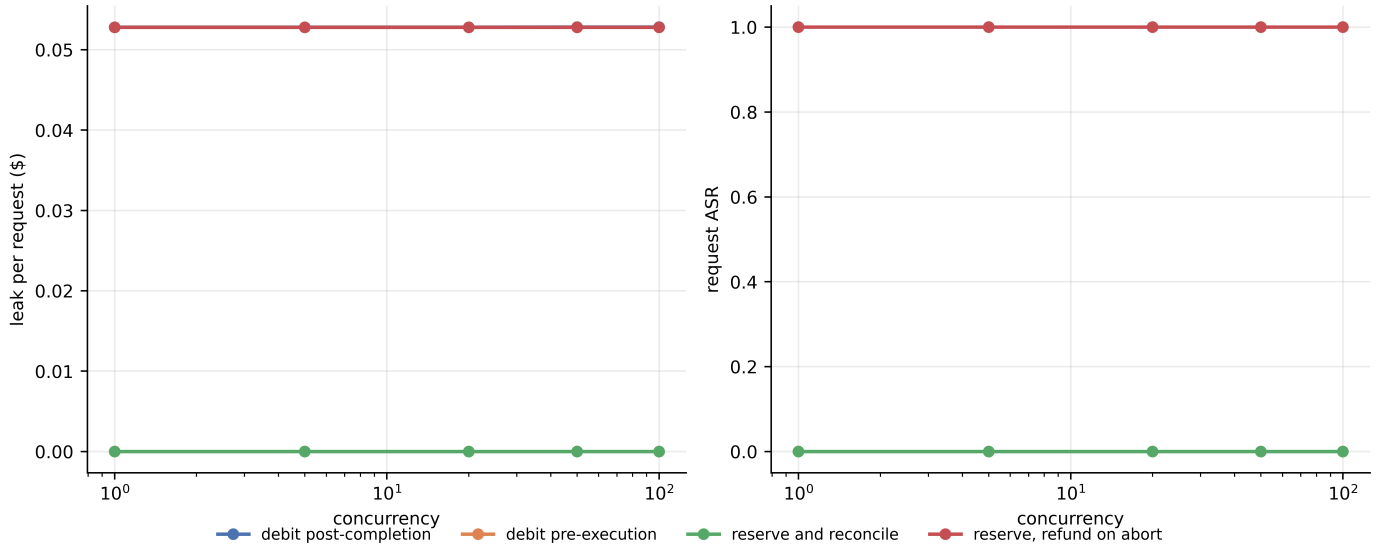


Figure 4: M1 negative result: mean leakage per request and request-level attack success against concurrency (1 to 100, log scale, volume fixed at 20 requests/cell). Both are invariant across the range, the empirical basis for treating commitment timing as per-request rather than contention-driven.

9. Defenses and Sufficiency Conditions

The ablation shows which primitive is load-bearing empirically. Here we state, per mechanism, conditions sufficient within the accounting model to guarantee Equation (1). These are model-level arguments, not universal theorems about deployed systems, and we keep what the model forces separate from what our implementation happens to supply.

M1, commitment timing. Two conditions suffice. (M1-a) Terminal-path totality: every terminal path (client disconnect, server exception, stream truncation) reaches a state where an accounting event for r is committed and durable. (M1-b) Refund boundedness: every refund satisfies Equation (2). If a reservation was taken, (M1-b) gives $N(r) = \mathrm{Res}(r) - F(r) \geq V(r)$; if not, (M1-a) forces the terminal event to commit $D(r) = V(r)$. Solvency is a separate condition, (M1-c): reserve atomically before execution and refuse the request when the reservation fails. The no-reserve-settle ablation cell satisfies (M1-a) and (M1-b) but not (M1-c), and measured exactly that: integrity held while the balance went negative.

M2, usage authority. Two conditions suffice. (M2-a) Authority: the usage value used for billing derives from state the attacker cannot control, $U_{\mathrm{bill}} = U_{\mathrm{true}}$. (M2-b) Pricing monotonicity: P is non-decreasing in each category. Computing U_{true} is not sufficient. An architecture that computes it correctly, records it, and bills $P(U_{\mathrm{decl}})$ satisfies "the server recounts" while violating (M2-a), and leaks $P(U_{\mathrm{true}}) - P(T(U_{\mathrm{true}}))$.

B0, synchronization. (B0-a): the authorization decision must be atomically coupled to the economic commitment on the path that authorizes service (Section 4) – a condition about *where* in the pipeline the atomic operation sits, not merely that one exists. Concurrency is one way to break that coupling; delayed settlement (Section 10) is another, requiring no concurrency at all.

Implementation-specific caveats. The model says "committed and durable"; our implementation achieves it with a cancellation-shielded settlement task, and a framework that cancelled that task would satisfy the architecture on paper while violating (M1-a). The upstream architecture satisfies (M2-a) only because the provider is honest by assumption. And (B0-a) is a property rather than a mechanism: our backends obtain it by a conditional `UPDATE` and by an anchor-row lock respectively, which is what Section 10 tests. The asynchronous family's own durability caveat is separate from all three: the queued settlement event itself needed a durable record independent of the message queue, which Section 10.6 describes; that fix was validated against the specific failure points we injected (worker termination before, during, and after consumption; hard termination of the queue process; duplicate delivery) and not against arbitrary-scale distributed failure or a host restart.

9.1. Practical guidance

Table 6 condenses the analysis into what an operator can act on.

9.2. What the strongest M2 defense costs

We measure the M2 recount at three levels because they answer different questions, reporting the noise band alongside the measured ratios per standard concerns about benchmarking rigor [76] and measurement bias from innocuous setup choices [77].

First, the standalone tokenizer (Figure 6). Cost is linear in input length (log–log slope 0.999 to 1.14 with $R^2 \geq 0.998$), so it budgets as tokens times a per-token constant, roughly 0.7 μs/token for tiktoken and 2.5 μs/token for SentencePiece. At

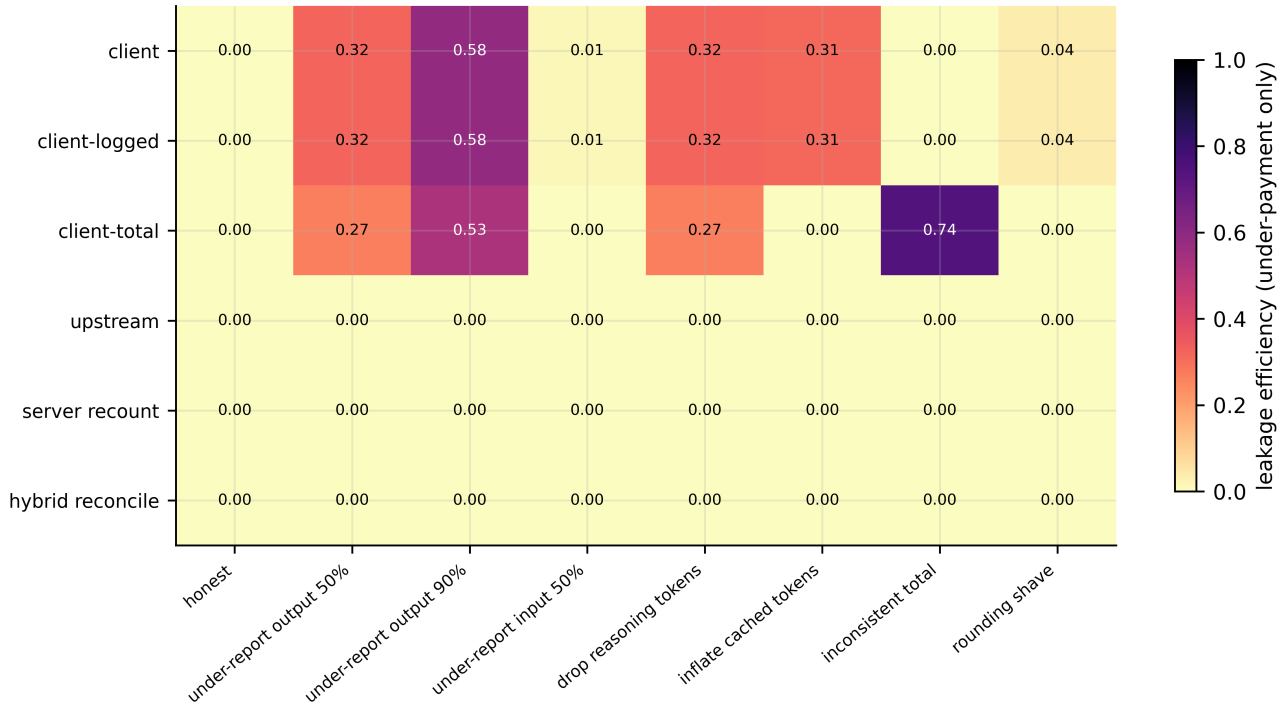


Figure 5: M2 leakage efficiency by architecture (rows) against declared-usage manipulation (columns), deterministic generator, medium tier, concurrency 1, 100 requests/cell. Server-authoritative rows are uniformly zero. The exploitable set differs between category-additive and flat-total pricing, the empirical form of Equation (3).

16,384 tokens the engines span 10.1 to 41.8 ms, a 2.9 to 3.6× implementation effect, and at fixed token count the workload alone changes cost by up to 3.4×.

Second, the gateway with the deterministic generator. Recount is nearly free at 1,024 tokens or fewer, retaining at least 0.96× throughput, but costs heavily at 16,384 tokens, at 0.05 to 0.76×. Tokenization time explains this, with Spearman rank correlation [78] $\rho = -0.868$ and $p = 1.3 \times 10^{-5}$.

Third, the local real-model experiment. With real autoregressive generation the recount is well under 0.1% of end-to-end time across 128-, 1024-, and 4096-token contexts: 0.014 to 0.019% at 64 new tokens, and 0.017 to 0.028% on an independent re-measurement with separately written timing code at 32 new tokens. The ratio rises as generation shortens, which is what the cost model predicts. Measured throughput ratios scatter around 1.0 with signs that flip across workloads, inside a ±10% noise band that is itself some 700× larger than the effect.

The apparent expense in the second case is an artifact of a generator that costs nothing: when generation is free, tokenization is the entire request. We report all three rather than the most convenient one, and we do not claim the recount is free. It is a real, predictable cost that is small relative to real inference in the tested configuration.

10. Cross-Architecture and Asynchronous Validation

A measurement study on one deployment invites the objection that its conclusions are deployment artifacts. We repeated the experiments while changing only the deployment, the storage architecture, or the inference data plane; the attack harness, the architectures under test, and the integrity gates stay identical across all runs. Each varied dimension (execution topology, accounting backend, inference data plane, settlement architecture) tests whether an observed behavior belongs to the architecture or to one implementation of it.

10.1. Execution topologies and accounting backends

Three topologies: a single uvicorn worker with no proxy, as control; nginx over four uvicorn workers, giving four independent event loops sharing one PostgreSQL and one Redis; and an nginx load balancer over two gateway containers. Across 96 accounting cells there were no mismatches against the control. Two genuine multi-worker defects surfaced during this work and are reported rather than hidden: the B0 runtime posture was per-process and would have reached one worker in four, and concurrent schema creation raced. Both were fixed.

Backend A keeps a mutable balance row and obtains atomicity from a conditional `UPDATE ... WHERE balance >= cost`. Backend B never mutates that row: every movement is an immutable entry in an append-only ledger and the balance is derived as the sum of entries, with the atomic reserve becoming a conditional append serialized on an anchor row.

All 17 comparison cells agreed, and we state plainly what that number contains. Only eight genuinely exercised both backends. Five agree for an analytic reason: the M2 billed amount is computed before any balance is read, so it cannot

Table 6: Practical defense summary. The model-level condition is what the accounting model requires; the implementation pattern is what we built and measured. Satisfying the pattern in a different system does not by itself establish the condition there.

Mech.	Failure mode	Property at risk	Model-level condition	Implementation pattern
B0	Authorization reads credit state not yet reflecting prior commitments	Ledger conservation, solvency	Authorization atomically coupled to economic commitment on the authorizing path	Conditional `UPDATE ... WHERE balance >= cost` evaluated on the request path, not deferred to a later worker
M1	A terminal path leaves the lifecycle without a committed debit, or over-refunds the unconsumed reservation	Accounting integrity; solvency separately	Terminal-path totality plus refund boundedness (Eq. 2); reservation for solvency	Cancellation-shielded settlement on every exit path, refund capped at $\mathrm{Res}(r) - V(r)$, atomic reserve before execution
M2	Correct usage is computed and stored, but billing reads a client-influenced representation	Accounting integrity	Billing basis derived from state outside attacker control; pricing monotone per category	Server-side recount used as the billing input, with the declared vector retained only for reconciliation
Async	Usage event lost, duplicated, or applied after the next authorization	Accounting integrity; conservation	Durable, idempotent accounting events with bounded reconciliation lag	PostgreSQL transactional outbox: settlement event committed durably in the same transaction as the accounting record, before reaching the queue; idempotency key per event; at-least-once delivery with duplicate suppression; queue-independent reconciliation recovers any event the queue fails to deliver

vary by backend. Four are B0 cells whose frozen debit module contains no reference to the backend abstraction, so they ran identical code twice. The defensible statement is that eight genuine backend-sensitive cases agreed and none disagreed. B0's independence from the storage representation is therefore untested, and we list it as a limitation.

10.2. A real serving stack

Every figure so far was produced against a generator we wrote. That is what makes the accounting arithmetic exact, but it leaves an obvious objection open, so we repeated the experiments with the token source replaced by llama.cpp's server [34] running SmolLM2-135M-Instruct [35] on CPU. This is self-hosted serving code we did not write, with a real subword tokenizer, real server-sent event streaming [21], and usage records the server produces itself. The gateway, architectures, accounting backends, settlement logic, and integrity gates are unchanged, and the deterministic default remains byte-identical, with both regression suites still passing.

Across 12 M1 cells and 48 M2 cells there were no safety disagreements. Vulnerable commitment-timing architectures still leak on abort and collapse to zero at completion, reserve-and-reconcile still leaks nothing, and every server-authoritative architecture still leaks exactly 0.000 across all eight manipulations.

Attack effectiveness did move, and we report it rather than average it away. Output under-reporting at 90% yields 0.690 here against 0.583 on the deterministic generator, because SmolLM2 emits no reasoning tokens and output therefore dominates the cost. For the same reason the K2 manipulation that drops hidden reasoning tokens becomes completely inert: there is nothing to drop. Under flat-total pricing three cells flip sign, because on this token mix flat-total overcharges by so much that even a 90% under-declared total still exceeds the authoritative category cost. Five of the 48 cells changed effectiveness in total; the per-cell comparison against the deterministic corpus is in the released artifact. The architectural conclusion is generator-independent; the per-manipulation magnitudes are not.

Real serving also exhibits a property the deterministic generator cannot: the server reports a prefix-cache split, and because cached input is priced an order of magnitude below uncached input, the honest charge for a byte-identical request depends on server cache state. Repeating identical requests, the reported split moved (24 then 52 of 53 prompt tokens in one instance) and the authoritative cost moved with it by 22 to 27%, reaching 32% at the extreme, on all three prompts tested. This sharpens the threat model – a client cannot verify its own bill even in principle, and over-declaring cached tokens carries genuine plausible deniability since the true value legitimately varies – though it is one stack with one prefix-cache implementation, not generalized to hosted APIs.

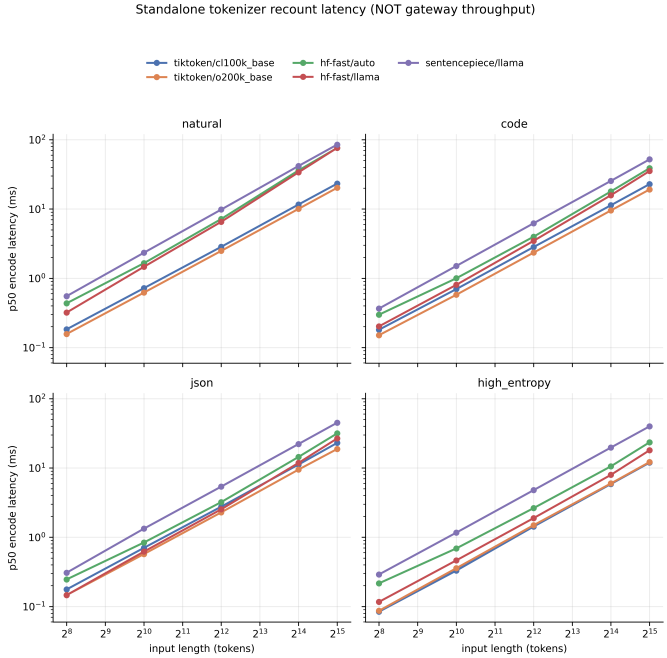


Figure 6: Standalone tokenizer recount latency (p50, log–log axes) by engine and input length, across four text workloads (natural prose, source code, JSON, and high-entropy/random text), over 100 cells whose token lengths were verified exactly. Cost is linear in input length within every workload, so recount budgets as tokens times a per-token constant; the workload shifts the constant, not the linear shape. This is a tokenizer microbenchmark, not a gateway throughput result: no request path, database, or network is involved.

10.3. Asynchronous accounting

Both backends so far settle before the response completes. Real metering pipelines often do not: the gateway emits a usage event and a separate worker applies it later. We therefore added a third accounting family, an event queue with a continuously running worker and a controlled reconciliation delay D, because a claim of architectural generality that quietly assumes strong consistency is not worth much.

With an honest pipeline, asynchronous accounting is delayed rather than lossy. The mean exposure window between value delivery and durable debit tracks D closely: at $D = 0, 10, 50, 100, 500$ ms the measured windows are 14.0, 14.9, 54.3, 103.7, and 504.1 ms, and the leak reconciles to exactly zero. Duplicated events are suppressed idempotently across all 25 injected duplicates. A lost event, by contrast, leaks permanently at every delay. This is an injected application-level fault (the event is never created), distinct from the infrastructure-level queue failure discussed in Section 10.6, and the result above is unaffected by that section's fix. Whether a straggler looks lost or merely late is a property of the observation horizon rather than of the architecture; we used four times the configured delay, floored at 400 ms, and the released artifact records the per-cell outcome. The reconciliation delays used here are injected experimental parameters, not delays observed in a production pipeline.

Table 7: B0 under deferred settlement with strictly sequential arrivals 20 ms apart, a two-request budget. No two requests are ever in flight together; over-serving here is not a concurrency race.

Settlement delay D	Requests over-served
0 ms	0
10 ms	0
50 ms	0
100 ms	1
500 ms	4

10.4. B0 revisited: the condition is not fundamentally about concurrency

The asynchronous family changed one of our own conclusions, and it is the sharpest systems result in the paper.

We issued requests strictly sequentially, 20 ms apart, so that no two were ever in flight together. We use "sequential" throughout this paper in exactly this sense: arrivals do not overlap, but settlement may still be pending when the next request is authorized. Section 10.5 introduces a deliberately different control, strict serialization of authorization and settlement, which additionally waits for settlement confirmation and is not interchangeable with the sequential arrivals studied here. Any over-serving in this experiment therefore does not arise from overlapping in-flight requests. Over-serving nonetheless appears once D reaches 100 ms, and reaches four requests over a two-request budget at $D = 500$ ms, with the balance driven negative (Table 7).

The mechanism is simple once seen. Authorization reads a committed balance that is correct but stale, because the debits for earlier requests are still in the queue. An atomic guarded decrement, the entire B0 defense, provides nothing if it lands after the next authorization has already read the balance. Concurrency was never the essential ingredient; it was one way of separating the check from the commitment. Delayed settlement is another, and it needs no concurrency at all.

We therefore restate the B0 condition as given in Section 9: authorization must be atomically coupled to economic commitment on the path that authorizes service. This is strictly more general than the concurrency framing we started with, and the earlier framing was too weak rather than wrong.

M2, by contrast, is untouched: leakage efficiency is identical at 0.5823 at every delay, confirming the analytic independence of usage authority from settlement timing.

10.5. Network-separated validation

Every topology in Section 10 so far ran on a single host over loopback, which leaves open whether Section 10.4's result is an artifact of that communication path. We repeated the asynchronous B0 experiment with the gateway on one machine and the accounting store, Redis, and the settlement worker on a second, reached over the real internet (a cloud VM in a different data center than the gateway host), with the attack harness, architectures, and integrity gates unchanged.

Table 8: Sequential arrivals with deferred settlement over the network-separated two-machine topology, budget $k = 2$, $N = 6$. Over-served counts across 5 repetitions per delay.

Settlement delay D	Over-served (5 reps)
0–2000 ms	0, 0, 0, 0, 0
5000 ms	3, 2, 2, 2, 2
10000 ms	4, 4, 4, 4, 4

While building this topology we found that our own sequential-arrival harness had a pacing confound: it paced requests by sleeping a nominal interval after each full round trip, and over the real network the round trip itself, not the sleep, dominated the achieved inter-arrival gap, measured directly from request timestamps at 513–857 ms rather than the intended 20 ms. We fixed the harness to record the achieved gap per request and reran with a delay range chosen to straddle that measured gap. The corrected sweep (Table 8, $N = 6$, budget $k = 2$, 5 repetitions per delay) shows a clean transition: no over-serving through $D = 2000$ ms, partial over-serving at $D = 5000$ ms, and saturation at $N - k = 4$ by $D = 10000$ ms.

The B0 mechanism persisted under a network-separated two-machine deployment, while strict serialization of authorization and settlement produced no observed over-serving across the tested settlement delays. We distinguish the two deliberately: *sequential arrivals with deferred settlement* (the mechanism above, and in Section 10.4) means requests do not overlap but settlement may still be pending when the next request is authorized; *strict serialization of authorization and settlement* means the harness confirms, from the account's own authoritative balance, that settlement for request i has completed before request $i+1$ is dispatched. The strict-serial control polls the authoritative balance until it equals the expected post-settlement value; an earlier implementation polled the queue's reported depth instead and was replaced after we found a transient window in which the queue reports empty while an event is still in transit. Over the same two-machine topology, strict serialization produced zero over-serving in every one of twelve tested cells ($D \in \{0, 500, 2000, 10000\}$ ms, 3 repetitions each; served equaled the budget exactly in all twelve).

This is evidence for one tested topology, one cloud region, and one host pair, not a characterization of networked deployments in general; Section 13 states the remaining scope precisely.

10.6. Durability of the queued settlement event

Fault injection during the network-separated work surfaced a second, independent finding about the testbed's own queue, distinct from the B0 admission race above. Before this fix, the only record of a pending settlement event was the message in the Redis queue; no durable record existed until the worker applied the debit. We tested this directly, not by simulation: we queued an event with its delay not yet elapsed, confirmed it was queued, hard -terminated the Redis process, restarted it, and found the event permanently lost, with the queue empty and the corresponding debit never applied.

The fix is a PostgreSQL transactional outbox, described as an implementation pattern in Table 6: the settlement event is written as a durable row in the same database transaction as the accounting record, committed before the event is handed to the queue, so a durable record exists independent of the queue from the moment the request is admitted. A settlement worker claims and applies an event in one atomic transaction, so no intermediate state exists in which an event is marked settled without the debit having landed. A queue-independent reconciliation pass scans for due, unsettled events and applies them directly from the durable record, recovering anything the queue failed to deliver.

We re-ran the targeted failure points this fix addresses and confirmed each by raw evidence, not inference: a worker terminated before an event was due recovered correctly on restart with exactly one debit applied; a hard termination of the queue process after an event was queued but not yet due was recovered by the reconciliation pass once the event's delay elapsed, rather than lost; duplicate delivery was suppressed with zero double-charging; and ledger conservation and final-balance correctness held in every case tested. We did not force a live database transaction rollback (argued analytically from the single-transaction design instead) and did not test a virtual-machine or host restart. Delivery remains at-least-once, not exactly-once: the queue or the reconciliation pass may both attempt to apply the same event, and the atomic claim ensures duplicate delivery does not result in a duplicate debit, which is a narrower and verifiable guarantee than claiming no event is ever delivered twice.

We characterize this architecture precisely rather than with a single adjective: before the fix it was process-crash vulnerable to termination of the queue process (a permanent loss, not merely a recoverable delay); after the fix it is eventually durable with at-least-once delivery and idempotent settlement application, at the specific failure points tested, and not validated against a host restart, arbitrary-scale distributed failure, or any topology beyond Section 10.5's own two-machine pair. The fix changes nothing about the B0 admission race studied throughout this paper – the race is in the authorization read, which it does not touch – and every pre-existing asynchronous-family experiment (Sections 10.3, 10.4) reproduced identical results when re-run against it, since none inject a queue failure.

11. Economic and Pricing Analysis

The security question in M2 reduces to an economic one. With U the true usage vector, T the attacker's transformation, and P the pricing function, a manipulation is profitable exactly under Equation (3): the pricing basis must read a field that T changes.

That has an immediate structural consequence. Under category-additive pricing, $P(U) = \sum_i p_i u_i$, a manipulation pays off if and only if it reduces a category the function actually reads. Declaring a total inconsistent with the subtotals is therefore inert, measured at 0.000, because category pricing never reads the declared total. Under flat-total pricing, $P(U) = p \cdot \text{total}(U)$, the same manipulation becomes the strongest attack in the catalog at 0.741, while reclassifying output into a

Table 9: Published list prices for one commercial provider, as an example of category-differentiated pricing in current practice [79]. Reproduced for context only; this paper makes no claim about any provider's metering or accounting implementation, and no commercial service was tested.

Model	Input ($/Mtok)	Cached input ($/Mtok)	Output ($/Mtok)	Cached : input
Sonnet 5	2	0.20	10	0.10
Haiku 4.5	1	0.10	5	0.10
Opus 5.5	4	0.20	20	0.05

discounted cached category becomes inert because no category is read at all. Mixed pricing over input, output, cached, and reasoning categories sits between the two, and its exploitable set is determined by which categories carry distinct prices.

Two kinds of price change must be distinguished. Uniform scaling of every price by a constant leaves leakage efficiency unchanged (confirmed across a 10× scaling in the full price-tier sensitivity table, released in the artifact), which is why we use efficiency as the headline metric and label absolute dollars as scenario values. Structural repricing – changing the ratios between categories, or moving between category-additive and flat-total bases – changes which transformations are profitable, and can change them qualitatively: the real-stack experiment demonstrates this directly, where three flat-total cells reverse sign on a token mix with no reasoning tokens because the pricing basis overcharges by more than the manipulation evades.

The pricing tiers used in our experiments are synthetic; Table 9 shows one provider's published list prices only to establish that such wide, category-differentiated ratios are current commercial practice, not a construction of ours.

We deliberately stop short of a revenue-impact model: estimating aggregate exposure would require assuming an attacker population and request volume, and the result would be invention dressed as measurement. What the analysis supports is a design rule: the pricing basis determines the attack surface and should be chosen with that in mind, not for billing convenience alone.

12. Discussion

Three dimensions of one accounting boundary behave differently because each is defective in a different element of the lifecycle: B0 is a shared-state property that fails whenever the check is separated from the commitment, while M1 and M2 are per-request properties with no concurrency dependence in the tested design (Sections 5 and 8).

The practical consequences are three: an operator cannot use traffic level as a proxy for exposure, since two of the three dimensions do not scale with load at all and the third fails under sequential arrivals if settlement lags; integrity and solvency need different controls, so a system with only one will look correct in whichever dimension it is measured; and in the evaluated architectures a correct recount did not prevent leakage unless authoritative for billing. The defenses themselves are ordinary; the contribution is knowing which one is load-bearing, and what it costs.

Engineering implications. Within the tested and modeled conditions, the results place each control at a specific architectural location. Authorization belongs on the same path as the economic commitment it depends on, not on a deferred worker, because an atomic decrement that lands after the next authorization has read the balance provides nothing. The billing input belongs to state outside attacker control, so the recount that feeds billing must be authoritative rather than merely computed. Every terminal path of the request lifecycle must reach a durable accounting event – a cancellation-shielded settlement task for M1, and the transactional outbox of Section 10.6 for the asynchronous family's queued event. The implementation patterns we built are not equivalent to the model-level conditions: satisfying a pattern in another system does not by itself establish the condition there, which is why Table 6 keeps them apart. These are implications for engineering metered services under the stated accounting model, demonstrated on LLM inference, not claims about deployed systems we did not test.

Relation to known weakness classes. This is the objection we consider most likely, so we state it ourselves: the individual mechanisms have precedents in isolation (Sections 3 and 3.5), B0 is a known race presented as a baseline, not a contribution, and we claim no novelty at the primitive level.

Beyond these known primitives, the paper adds the unification stated in the contributions list (Section 1), plus two things no primitive-level description predicts: the M1 ablation shows, by exhaustive model checking rather than by example, that solvency and accounting integrity are orthogonal and need different primitives, which "there is a streaming bug" does not predict; and the M2 ablation shows an architecture that recounts usage correctly can still leak if billing is not authoritative. We do not assert practitioners believed a recount was sufficient; we show experimentally that in the evaluated architectures it was not.

13. Threats to Validity

We organize threats as internal, construct, external, and formal validity [80, 81], and add one for the degree of physical distribution actually exercised.

Internal. A single uvicorn worker in the base configuration makes gateway recount numbers an upper bound. The host is shared and Docker scheduling is not controlled. Local loopback adds a roughly 45 ms floor per request on this configuration, which makes the gateway ratios conservative for small inputs. Leakage that depends on delivered tokens is sensitive to disconnect timing at high concurrency: the within-cell standard deviation of delivered tokens is exactly zero at concurrency 20 or below and about 0.011 at 50 and 100.

Construct. The B0 debit path is a frozen module predating the pluggable accounting backends and does not reference them, so backend parity covers M1 and M2 only and B0's storage independence is untested. Our detectability instrumentation restates experimenter-assigned labels rather than measuring evidence, so we make no detectability claim.

Formal. Evidence labels, proof scope, and what neither specification claims are stated in Section 7 and apply here without repetition. The primary specification is finite (two to three concurrent requests, two value chunks, unit pricing) and assumes strongly consistent accounting state, so it does not cover the asynchronous family. The second specification (Section 7.6) covers the asynchronous-settlement mechanism as an untimed, causal-ordering abstraction, with no notion of milliseconds, network behavior, or database semantics, checked at two finite (N, k) points. Neither specification models multi-category pricing or tokenizer disagreement.

External. One serving stack, one small model on CPU, one prompt family. This establishes that the architectural conclusions are not artifacts of our own generator; it does not establish behavior on commercial providers, and the cache-split nondeterminism is a property of this stack's prefix cache rather than a claim about hosted APIs. The generator-dependent cells we report bound how far per-manipulation magnitudes travel. Reconciliation delays are injected rather than observed in production, so the asynchronous result is conditional and stated as such: if settlement lags authorization by more than the inter-arrival time, atomicity on the settlement path does not prevent overserving. Pricing tiers are synthetic. More broadly, the experiments establish that the modeled accounting failures are realizable under the specified service architectures and workloads; they do not establish that any particular commercial provider implements these architectures, that the mechanisms are prevalent across deployed LLM services, or that a specific provider is vulnerable. Public issue and pull-request evidence cited in this paper shows only that related accounting and settlement behaviors are reported in deployed and open-source infrastructure, not a claim about any named system's current state.

Distribution. Separation in the topology experiments of Section 10 (execution topologies, accounting backends) is temporal, not physical: those topologies ran on a single host under Docker Compose over loopback. The asynchronous settlement family was additionally run with the accounting store, Redis, and the settlement worker on a second machine reached over the real internet (Section 10.5), and the condition persisted there, establishing that it is not an artifact of loopback communication specifically. This narrows but does not remove the limitation: we tested one cloud-region host pair, not network partition, clock skew, or cross-machine queueing behavior in general, and we did not test a VM or host restart, a wide-area or multi-region topology, or a production deployment.

14. Responsible Disclosure and Ethics

This work produced no new vulnerability findings against any third-party, live, or production system. All experiments ran against the local testbed shipped with the released artifact, whose harness defaults to `localhost` and contains no third-party endpoints. No credentials, real user data, or production billing APIs were used. The public artifacts cited as prior evidence in Section 3.5 were public, and in two cases already fixed, before this work; they are reports written by those projects' own maintainers and users, not findings of ours, and we reviewed only their published issue and pull-request pages. Every attack in the artifact is paired with a defense, and the testbed is released so that operators can measure their own gateways.

15. Conclusion

This paper is an evidence-backed study of accounting integrity in metered computational services, carried out on LLM inference serving as the concrete workload. Client-side underpayment is best modeled as one accounting boundary with three placements: synchronization, commitment timing, and usage authority. Localizing each to a state transition predicts, and our measurements confirm, that integrity and solvency are closed by different primitives and that a server-side recount must be authoritative rather than merely computed. Exhaustive model checking establishes the M1 orthogonality over all four combinations rather than the two we implemented, and the same checking refuted our first specification before it confirmed the corrected one.

The result we did not expect concerns the baseline. B0 began, in this implementation, as a concurrency race, and the asynchronous experiment showed that concurrency was incidental to the underlying failure: what matters is whether authorization is atomically coupled to economic commitment on the path that authorizes service. Delayed settlement breaks that coupling with no concurrency whatsoever. The corrected condition is more general than the one we started with, and it is the form an operator should test against. In the tested configuration the strongest M2 defense costs a small and predictable fraction of real inference time; B0's atomic decrement and M1's reserve-reconcile path are not characterized by this same cost measurement.

Acknowledgements

No external acknowledgements apply to this work.

CRediT authorship contribution statement

Hardik: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Validation, Visualization, Writing – original draft, Writing – review and editing. **Natasha Kaila**: Investigation, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data availability

Source code, the attack harness, defense implementations, the TLA+ specification, and all raw experimental data are released at `https://github.com/meowlawat/TOKEN-ACCOUNTING-INTEGRITY-IN-LLM-METERING`, tagged release `v1.1.0`, which is the version reported in this paper, including the network-separated validation and strict-serialization control (Section 10.5), the transactional-outbox durability fix (Section 10.6), and the second, asynchronous-settlement formal model (Section 7.6). Tags `v1.0.0` and `v1.0.1` are preserved unchanged as earlier historical snapshots that precede this manuscript's additions and do not contain them; `v1.0.1` differs from `v1.0.0` only in how the B0 condition is stated and in no measured value. The complete artifact (source code, experiment scripts, both TLA+ specifications, raw and processed results, figures, tables, and reproduction scripts) corresponding to this version is archived in Zenodo under the concept DOI 10.5281/zenodo.22085827, which always resolves to the most recent archived version, and under version DOI 10.5281/zenodo.23261555 specifically, the version corresponding to GitHub tag `v1.1.0`. An earlier version DOI, 10.5281/zenodo.22086254, archives the artifact at tag `v1.0.1` and remains resolvable as the historical record. The deposited archive's contents and checksum are listed in `release/zenodo/` in the repository.

References

[1] Sun, G., Wang, Z., Tian, B., Liu, M., et al., 2025a. CoIn: Counting the invisible reasoning tokens in commercial opaque LLM APIs. arXiv:2505.13778.

[2] Sun, G., Wang, Z., Zhao, X., Tian, B., et al., 2025b. Invisible tokens, visible bills: The urgent need to audit hidden operations in opaque LLM services. arXiv:2505.18471.

[3] Hoque, S., Zhang, J., Sun, J., Suya, F., 2026. Token inflation: How dishonest providers can overcharge for large language model usage. arXiv:2605.30040.

[4] Kelly, D., Glavin, F.G., Barrett, E., 2021. Denial of wallet — Defining a looming threat to serverless computing. arXiv:2104.08031.

[5] Dorsett, M., Mann, S., Chowdhury, J., Mahmood, A., 2025. A comprehensive review of denial of wallet attacks in serverless architectures. arXiv:2508.19284.

[6] OWASP GenAI Security Project, 2025. LLM10:2025 unbounded consumption. OWASP Top 10 for LLM Applications. `https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/`.

[7] Sysdig Threat Research Team, 2024. LLMjacking: Stolen cloud credentials used in new AI attack. Sysdig. `https://www.sysdig.com/blog/llmjacking-stolen-cloud-credentials-used-in-new-ai-attack`.

[8] Guan, Y., 2026. AEX: Non-intrusive multi-hop attestation and provenance for LLM APIs. arXiv:2603.14283.

[9] Wang, F., Tian, Z., 2026. Evidence-bound gateway-path provenance for third-party LLM inference. arXiv:2606.22560.

[10] MITRE, n.d. CWE-807: Reliance on untrusted inputs in a security decision. Common Weakness Enumeration, version 4.20. `https://cwe.mitre.org/data/definitions/807.html`.

[11] QuantumNous, 2026. Streaming responses requests may refund all pre-consumed quota when final usage is missing after client disconnect. new-api issue #5235. `https://github.com/QuantumNous/new-api/issues/5235`.

[12] Lightning Labs, 2026. pricesrpc+proxy: Metered draw-down of prepaid L402 tokens, with a reference pricer daemon. Aperture pull request #247 (merged). `https://github.com/lightninglabs/aperture/pull/247`.

[13] Kettle, J., 2023. Smashing the state machine: The true potential of web race conditions. PortSwigger Research. `https://portswigger.net/research/smashing-the-state-machine`.

[14] Wang, R., Chen, S., Wang, X., Qadeer, S., 2011. How to shop for free online – security analysis of cashier-as-a-service based web stores, in: 2011 IEEE Symposium on Security and Privacy, pp. 465–480. `https://doi.org/10.1109/SP.2011.26`.

[15] Warszawski, T., Bailis, P., 2017. ACIDRain: concurrency-related attacks on database-backed web applications, in: Proceedings of the 2017 ACM International Conference on Management of Data, pp. 5–20. `https://doi.org/10.1145/3035918.3064037`.

[16] Armbrust, M., Fox, A., Griffith, R., Joseph, A.D., et al., 2010. A view of cloud computing. Communications of the ACM 53, 50–58. `https://doi.org/10.1145/1721654.1721672`.

[17] Bass, L., Clements, P., Kazman, R., 2021. Software Architecture in Practice. Fourth edition, Addison-Wesley, Boston.

[18] Shumailov, I., Zhao, Y., Bates, D., Papernot, N., Mullins, R., Anderson, R., 2020. Sponge examples: Energy-latency attacks on neural networks. arXiv:2006.03463.

[19] Saltzer, J., Schroeder, M., 1975. The protection of information in computer systems. Proceedings of the IEEE 63, 1278–1308. `https://doi.org/10.1109/PROC.1975.9939`.

[20] O'Neil, P.E., 1986. The escrow transactional method. ACM Transactions on Database Systems 11, 405–430.

[21] WHATWG, n.d. Server-sent events. HTML Living Standard. `https://html.spec.whatwg.org/multipage/server-sent-events.html`.

[22] Fielding, R., Nottingham, M., Reschke, J., 2022. HTTP semantics. RFC 9110, IETF.

[23] Berenson, H., Bernstein, P., Gray, J., Melton, J., et al., 1995. A critique of ANSI SQL isolation levels, in: Proceedings of the 1995 ACM SIGMOD international conference on Management of data - SIGMOD '95, pp. 1–10. `https://doi.org/10.1145/223784.223785`.

[24] Adya, A., Liskov, B., O'Neil, P., 2000. Generalized isolation level definitions, in: Proceedings of 16th International Conference on Data Engineering (Cat. No.00CB37073), pp. 67–78. `https://doi.org/10.1109/ICDE.2000.839388`.

[25] Garcia-Molina, H., Salem, K., 1987. Sagas, in: Proceedings of the 1987 ACM SIGMOD international conference on Management of data - SIGMOD '87, pp. 249–259. `https://doi.org/10.1145/38713.38742`.

[26] Helland, P., 2012. Idempotence is not a medical condition: an essential property for reliable systems. Queue 10, 30–46. `https://doi.org/10.1145/2181796.2187821`.

[27] Bailis, P., Fekete, A., Franklin, M.J., Ghodsi, A., et al., 2014. Coordination avoidance in database systems. Proceedings of the VLDB Endowment 8, 185–196. `https://doi.org/10.14778/2735508.2735509`.

[28] Herlihy, M.P., Wing, J.M., 1990. Linearizability: a correctness condition for concurrent objects. ACM Transactions on Programming Languages and Systems 12, 463–492. `https://doi.org/10.1145/78969.78972`.

[29] Vogels, W., 2009. Eventually consistent. Communications of the ACM 52, 40–44. `https://doi.org/10.1145/1435417.1435432`.

[30] Kleppmann, M., 2017. Designing Data-Intensive Applications. O'Reilly Media, Sebastopol.

[31] Sennrich, R., Haddow, B., Birch, A., 2015. Neural machine translation of rare words with subword units. arXiv:1508.07909.

[32] Kudo, T., Richardson, J., 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. arXiv:1808.06226.

[33] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., et al., 2023. Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), pp. 611–626.

[34] Gerganov, G., et al., n.d. llama.cpp: LLM inference in C/C++ [Software]. `https://github.com/ggml-org/llama.cpp`.

[35] Ben Allal, L., Lozhkov, A., Bakouch, E., Martín Blázquez, G., et al., 2025. SmolLM2: When smol goes big — Data-centric training of a small language model. arXiv:2502.02737.

[36] OpenAI, n.d. tiktoken: A fast BPE tokeniser for use with OpenAI's models [Software]. `https://github.com/openai/tiktoken`.

[37] Agrawal, A., Kedia, N., Panwar, A., Mohan, J., et al., 2024. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve.

arXiv:2403.02310.
[38] Zhong, Y., Liu, S., Chen, J., Hu, J., et al., 2024. DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. arXiv:2401.09670.
[39] Zheng, L., Yin, L., Xie, Z., Sun, C., et al., 2023. SGLang: efficient execution of structured language model programs. arXiv:2312.07104.
[40] Leviathan, Y., Kalman, M., Matias, Y., 2022. Fast inference from transformers via speculative decoding. arXiv:2211.17192.
[41] Petrov, A., La Malfa, E., Torr, P.H.S., Bibi, A., 2023. Language model tokenizers introduce unfairness between languages. arXiv:2305.15425.
[42] Lamport, L., 1994. The temporal logic of actions. ACM Transactions on Programming Languages and Systems 16, 872–923.
[43] Lamport, L., 2002. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, Boston.
[44] Yu, Y., Manolios, P., Lamport, L., 1999. Model checking TLA+ specifications, in: Correct Hardware Design and Verification Methods (CHARME), LNCS 1703, pp. 54–66.
[45] Clarke, E.M., Emerson, E.A., Sistla, A.P., 1986. Automatic verification of finite-state concurrent systems using temporal logic specifications. ACM Transactions on Programming Languages and Systems 8, 244–263. https://doi.org/10.1145/5397.5399.
[46] Holzmann, G., 1997. The model checker SPIN. IEEE Transactions on Software Engineering 23, 279–295. https://doi.org/10.1109/32.588521.
[47] Jackson, D., 2002. Alloy: a lightweight object modelling notation. ACM Transactions on Software Engineering and Methodology 11, 256–290. https://doi.org/10.1145/505145.505149.
[48] Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M., 2015. How Amazon Web Services uses formal methods. Communications of the ACM 58, 66–73.
[49] Woodcock, J., Larsen, P.G., Bicarregui, J., Fitzgerald, J., 2009. Formal methods: practice and experience. ACM Computing Surveys 41, 1–36. https://doi.org/10.1145/1592434.1592436.
[50] Gleirscher, M., Marmsoler, D., 2020. Formal methods in dependable systems engineering: a survey of professionals from Europe and North America. Empirical Software Engineering 25, 4473–4546. https://doi.org/10.1007/s10664-020-09836-5.
[51] Lamport, L., 2015. Who builds a house without drawing blueprints?. Communications of the ACM 58, 38–41. https://doi.org/10.1145/2736348.
[52] Klein, G., Elphinstone, K., Heiser, G., Andronick, J., et al., 2009. seL4: formal verification of an OS kernel, in: Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles, pp. 207–220. https://doi.org/10.1145/1629575.1629596.
[53] Leroy, X., 2009. Formal verification of a realistic compiler. Communications of the ACM 52, 107–115. https://doi.org/10.1145/1538788.1538814.
[54] Hawblitzel, C., Howell, J., Kapritsos, M., Lorch, J.R., et al., 2015. IronFleet: proving practical distributed systems correct, in: Proceedings of the 25th Symposium on Operating Systems Principles, pp. 1–17. https://doi.org/10.1145/2815400.2815428.
[55] Bornholt, J., Joshi, R., Astrauskas, V., Cully, B., et al., 2021. Using lightweight formal methods to validate a key-value storage node in Amazon S3, in: Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, pp. 836–850. https://doi.org/10.1145/3477132.3483540.
[56] Fonseca, P., Zhang, K., Wang, X., Krishnamurthy, A., 2017. An empirical study on the correctness of formally verified distributed systems, in: Proceedings of the Twelfth European Conference on Computer Systems, pp. 328–343. https://doi.org/10.1145/3064176.3064183.
[57] Claessen, K., Hughes, J., 2000. QuickCheck: a lightweight tool for random testing of Haskell programs, in: Proceedings of the fifth ACM SIGPLAN international conference on Functional programming, pp. 268–279. https://doi.org/10.1145/351240.351266.
[58] Chen, T.Y., Kuo, F.-C., Liu, H., Poon, P.-L., et al., 2018. Metamorphic testing: a review of challenges and opportunities. ACM Computing Surveys 51, 1–27. https://doi.org/10.1145/3143561.
[59] Segura, S., Fraser, G., Sanchez, A.B., Ruiz-Cortes, A., 2016. A survey on metamorphic testing. IEEE Transactions on Software Engineering 42, 805–824. https://doi.org/10.1109/TSE.2016.2532875.
[60] Barr, E.T., Harman, M., McMinn, P., Shahbaz, M., et al., 2015. The oracle problem in software testing: a survey. IEEE Transactions on Software Engineering 41, 507–525. https://doi.org/10.1109/TSE.2014.2372785.
[61] Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., et al., 2016. Chaos engineering. IEEE Software 33, 35–41. https://doi.org/10.1109/MS.2016.60.
[62] Alvaro, P., Rosen, J., Hellerstein, J.M., 2015. Lineage-driven fault injection, in: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, pp. 331–346. https://doi.org/10.1145/2723372.2723711.
[63] Kingsbury, K., Alvaro, P., 2020. Elle: inferring isolation anomalies from experimental observations. Proceedings of the VLDB Endowment 14, 268–280. https://doi.org/10.14778/3430915.3430918.
[64] Gunawi, H.S., Hao, M., Suminto, R.O., Laksono, A., et al., 2016. Why does the cloud stop computing? Lessons from hundreds of service outages, in: Proceedings of the Seventh ACM Symposium on Cloud Computing, pp. 1–16. https://doi.org/10.1145/2987550.2987583.
[65] BerriAI, 2025. Usage data lost when streaming responses are terminated early by client disconnect. LiteLLM issue #14457 (open). https://github.com/BerriAI/litellm/issues/14457. Accessed 23 September 2026.
[66] Etalab, 2026. An interrupted streamed chat completion is neither billed nor released. OpenGateLLM issue #1148 (fixed). https://github.com/etalab-ia/OpenGateLLM/issues/1148. Accessed 23 September 2026.
[67] Router-for-me, 2026. /v1/responses streaming occasionally finishes with HTTP 200 but no usage returned, causing downstream billing to be 0. CLIProxyAPI issue #2179 (closed). https://github.com/router-for-me/CLIProxyAPI/issues/2179. Accessed 23 September 2026.
[68] Stol, K.-J., Fitzgerald, B., 2018. The ABC of software engineering research. ACM Transactions on Software Engineering and Methodology 27, 1–51. https://doi.org/10.1145/3241743.
[69] Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., et al., 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods 17, 261–272. https://doi.org/10.1038/s41592-019-0686-2.
[70] Seabold, S., Perktold, J., 2010. Statsmodels: econometric and statistical modeling with Python, in: Proceedings of the Python in Science Conference, pp. 92–96. https://doi.org/10.25080/Majora-92bf1922-011.
[71] Efron, B., 1979. Bootstrap methods: another look at the jackknife. The Annals of Statistics 7, 1–26. https://doi.org/10.1214/aos/1176344552.
[72] Arcuri, A., Briand, L., 2014. A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. Software Testing, Verification and Reliability 24, 219–250. https://doi.org/10.1002/stvr.1486.
[73] Collberg, C., Proebsting, T.A., 2016. Repeatability in computer systems research. Communications of the ACM 59, 62–69. https://doi.org/10.1145/2812803.
[74] Feitelson, D.G., 2015. From repeatability to reproducibility and corroboration. ACM SIGOPS Operating Systems Review 49, 3–11. https://doi.org/10.1145/2723872.2723875.
[75] Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., Appleton, G., et al., 2016. The FAIR guiding principles for scientific data management and stewardship. Scientific Data 3, article 160018. https://doi.org/10.1038/sdata.2016.18.
[76] Kalibera, T., Jones, R., 2013. Rigorous benchmarking in reasonable time, in: Proceedings of the 2013 international symposium on memory management, pp. 63–74. https://doi.org/10.1145/2464157.2464160.
[77] Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2009. Producing wrong data without doing anything obviously wrong!, in: Proceedings of the 14th international conference on Architectural support for programming languages and operating systems, pp. 265–276. https://doi.org/10.1145/1508244.1508275.
[78] Spearman, C., 1904. The proof and measurement of association between two things. The American Journal of Psychology 15, 72–101. https://doi.org/10.2307/1412159.
[79] Anthropic, 2026. Pricing. https://claude.com/pricing. Accessed 23 September 2026.
[80] Runeson, P., Höst, M., 2009. Guidelines for conducting and reporting case

study research in software engineering. Empirical Software Engineering 14, 131–164. https://doi.org/10.1007/s10664-008-9102-8.

[81] Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., et al., 2012. Experimentation in Software Engineering. Springer, Berlin, Heidelberg.